

DSP Lab-2

Transformations of the Time Variable for Discrete Time Signals

Important Signal Operations in Matlab:

1. Signal Scaling

Description:

In this operation, each sample of the signal is multiplied by a scalar constant.

2. Signal Shifting

Description:

During a shift operation a signal changes its position in time where each sample of $x(n)$ is shifted by an amount k to obtain a shifted sequence $y[n]$.

$$y(n) = \{x(n - k)\}$$

In Matlab, this operation is not so simple, both “x” and “n” need to be processed separately in order to get the shifted signal $x[n-k]$. Perform following steps to in Matlab to perform the signal shift.

- i) Check whether k is positive or negative?
- ii) If k is positive it means shift is towards right i.e. $x[n-k]$ then do following
 - a. As signal is shifting towards right time axes “n” should be extended towards right side by “k” samples i.e. if n is ending at n_2 then it should now end at $n_2 + k$.
 - b. Since the signal has now moved towards right meaning initial “k” samples in “x” are empty so concatenate “k” zeroes in the beginning of the sequence x.
- iii) If k is negative it means shift is towards left i.e. $x[n+k]$ then do following
 - c. As signal is shifting towards left time axes “n” should be extended

towards left side by “k” samples i.e. if n is starting at n1 then it should now start at n1 - k.

d. Since the signal has now moved towards left meaning last “k” samples in

“x” are empty so concatenate “k” zeroes in the end of the sequence x.

3. Folding

Description:

Folding operation is also termed as flipping a signal where each sample is of x[n] is mirrored around n=0 to obtain a folded sequence y[n] = x[-n].

In Matlab, folding operation can be performed in three steps

- i) Flip the magnitude sequence “x”
- ii) Flip the time index “n”
- iii) Now multiply the flipped time vector “n” with a minus

Note that in Matlab a built-in function “fliplr(m)” can be used to flip any sequence.

Differentiation in DSP

In **continuous time**, differentiation means:

$$y(t) = \frac{d}{dt}x(t)$$

In **discrete time**, we approximate the derivative by the **first difference**:

$$y[n] = x[n] - x[n - 1]$$

This is also called a **differentiator**.

- It highlights **changes** in the signal (edges, transitions).
- If the signal is constant, output = 0.

Accumulator in DSP

In **continuous time**, integration means:

$$y(t) = \int x(t) dt$$

In **discrete time**, the accumulator is the running sum:

$$\begin{aligned} y(n) &= \sum_{k=-\infty}^n x(k) = \sum_{k=-\infty}^{n-1} x(k) + x(n) \\ &= y(n-1) + x(n) \end{aligned}$$

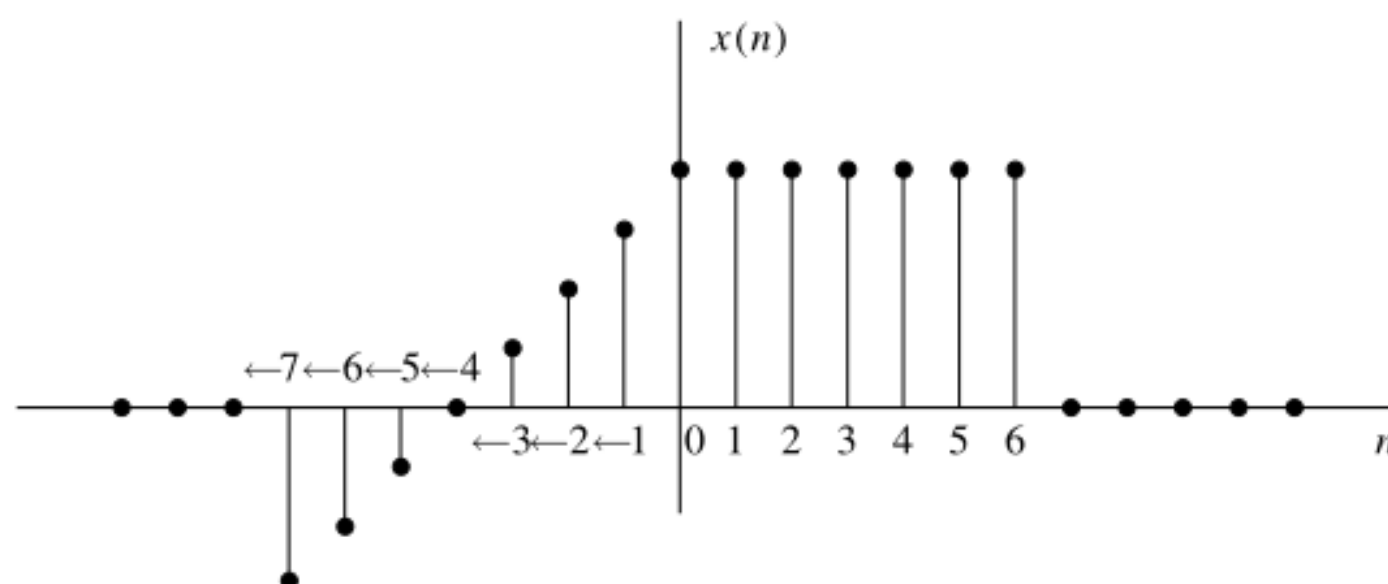
It's also called the **discrete-time integrator**.

- It keeps adding input values over time.
- Useful for signal smoothing or as part of IIR filters.

In Lab Task

Task-1:

The discrete-time signal $x[n]$ is shown in the figure below.



1. Define a discrete-time signal $x[n]$ and its index array **n**.
2. Apply the following operations:
 - a. $y1[n] = x[-n]$
 - b. $y2[n] = x[n-3]$
 - c. $y3[n] = x[n+2]$
 - d. $y4[n] = x[2n]$

- e. $y_5[n] = x[n/2]$
 - f. $y_2[n] = x[-2n+4]$
3. For each operation, generate the transformed signal and plot it along with the original signal using stem plots.

Task-2:

1. Consider the discrete-time signal

$$x(n) = \begin{cases} |n|, & -3 \leq n \leq 3 \\ 0, & \text{otherwise} \end{cases}$$

- (a) Implement a differentiator defined as:

$$y[n] = x[n] - x[n-1], \quad \text{with } x[-1] = 0$$

- (b) Implement an accumulator defined as:

$$z[n] = \sum_{k=0}^n x[k]$$

2. Plot the original signal **x[n]**, the differentiated signal **y[n]**, and the accumulated signal **z[n]** using **stem plots**.
3. Comment on the results:
- o How does the differentiator respond to changes in the signal?
 - o How does the accumulator modify the signal?

Task 3:

Given the discrete-time signal

$$x(n) = \{2, 4, 6, 8, 10\}, \quad 0 \leq n \leq 4$$

1. Plot the original signal using a stem plot.

2. Downsample the signal by a factor of 2.
3. Plot and compare both signals on the same axis.

Task 4:

Given the discrete-time signal

$$x(n) = \{2, 4, 6, 8, 10\}, \quad 0 \leq n \leq 4$$

1. Plot the original signal using a stem plot.
2. Upsample the signal by a factor of 2 by inserting zeros between the samples.
3. Plot and compare both signals on the same axis.

Post Lab Task

Record a short voice signal (5–10 seconds) using MATLAB or any audio recording tool and save it as a .wav file. Then:

1. Read and plot the recorded signal in MATLAB.
2. Apply amplitude scaling to the signal and plot the result.
3. Apply time reversal to the signal and plot the result.
4. Downsample the signal by a factor of 2 and plot the result.
5. Upsample the signal by a factor of 2 and plot the result.
6. Compare the original signal with all transformed versions using plots.
7. Play the original and transformed signals using the sound function and describe the differences you hear.